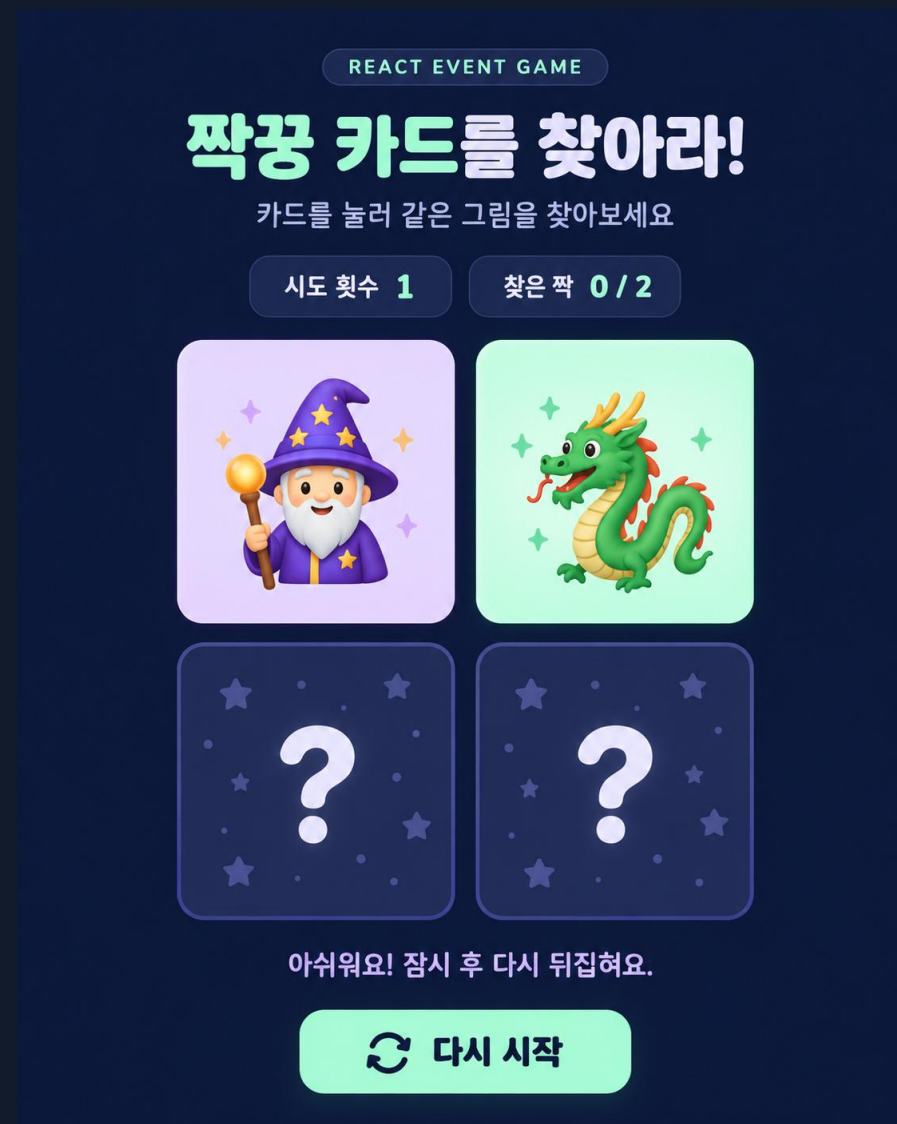


2_5 이벤트 속성 이해하기

모바일 메모리 카드 게임



이번 단계의 완성 화면

남색 배경과 게임 제목

시도 횟수 0, 찾은 짝 0 / 2

물음표 카드 4장을 2×2로 배치

누르면 살짝 작아지는 카드

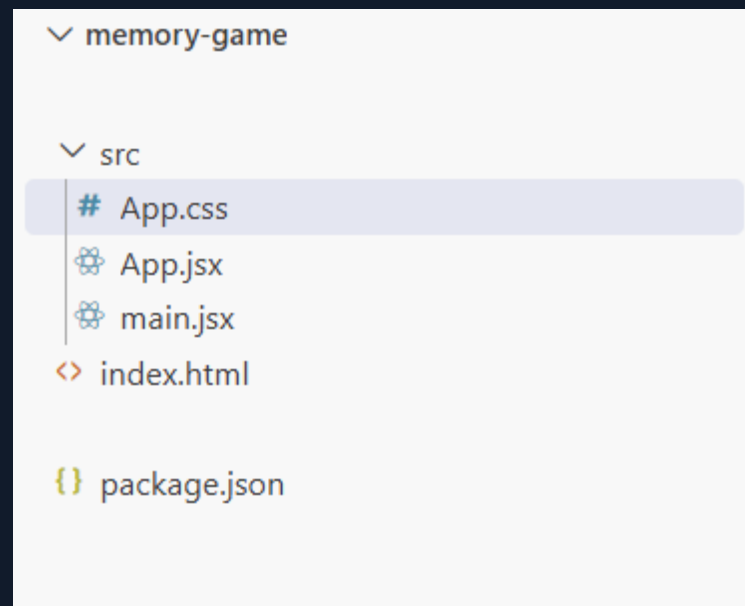
첨부한 실제 실행 화면이에요. 아직 카드를 눌러도 그림은 나오지 않아요.



01 프로젝트 폴더 만들기

memory-game 바로 아래
package.json
index.html

src 폴더 안
main.jsx
App.jsx
App.css



탐색기의 새 폴더로 src를 만들어요. App.jsx와 App.css는 같은 src 폴더에 두어요.

03 package.json 입력

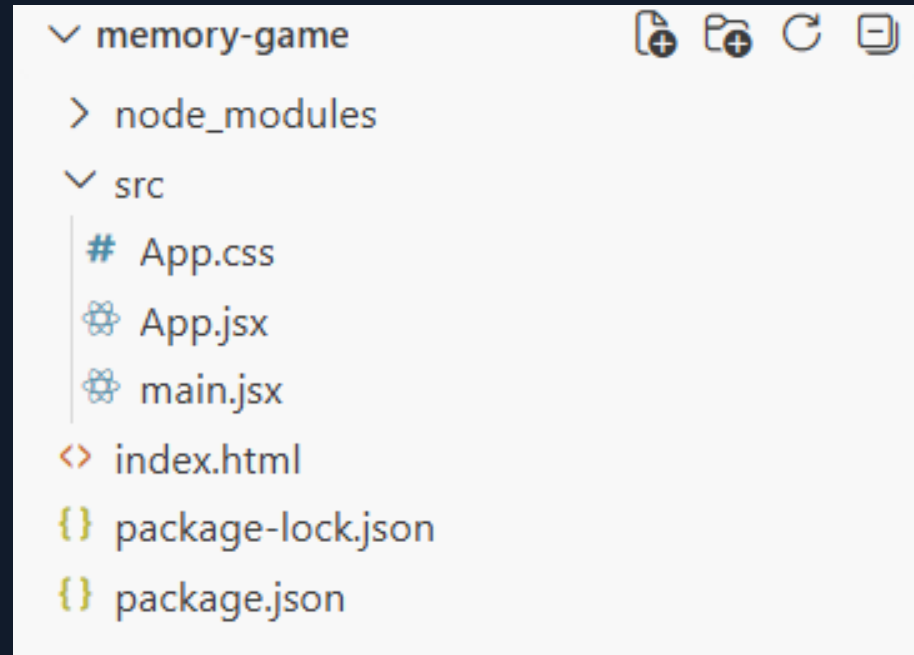
```
{  
  "name": "memory-game",  
  "version": "1.0.0",  
  "private": true,  
  "type": "module",  
  "scripts": {  
    "dev": "vite"  
  }  
}
```

입력 후 Ctrl + S로 저장해요. JSON에는 주석을 넣지 않아요.

04 React와 Vite 설치

`npm install react react-dom`

`npm install -D vite`



memory-game 터미널에서 한 줄씩 실행해요. 설치가 끝난 뒤 다음 명령을 입력해요.

05 index.html 입력

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport"
      content="width=device-width, initial-scale=1.0" />
    <title> 짝꿍 카드 찾기 </title>
  </head>
  <body>
    <div id="root"> </div>
    <script type="module" src="/src/main.jsx"> </script>
  </body>
</html>
```

viewport 설정으로 휴대폰 너비에 맞춰 표시해요. 입력 후 저장하세요.

06 main.jsx 입력

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

const root = createRoot(
  document.getElementById("root")
);

root.render(<App />);
```

입력 후 저장해요. App.jsx의 화면을 index.html 안의 root에 연결해요.

07 App.jsx 1/3

```
import React from "react";
import "./App.css";
export default function App() {
  return (
    <main className="game">
      <header className="game-header">
        <p className="eyebrow">MEMORY GAME</p>
        <h1>짝꿍 카드를 찾아라!</h1>
        <p className="description">
          같은 그림의 카드 두 장을 찾아보세요.
        </p>
      </header>
```

세 부분을 모두 이어 붙인 뒤 Ctrl + S로 저장해요. 중간에는 문법 오류가 보일 수 있어요.

07 App.jsx 2/3

```
<div className="scoreboard">
  <p>시도 횟수 <strong>0</strong> </p>
  <p>찾은 짝 <strong>0 / 2</strong> </p>
</div>
<section className="card-grid" aria-label="게임 카드">
  <button type="button" className="card"
    aria-label="1번 카드">?</button>
  <button type="button" className="card"
    aria-label="2번 카드">?</button>
  <button type="button" className="card"
    aria-label="3번 카드">?</button>
  <button type="button" className="card"
    aria-label="4번 카드">?</button>
```

세 부분을 모두 이어 붙인 뒤 Ctrl + S로 저장해요. 중간에는 문법 오류가 보일 수 있어요.

07 App.jsx 3/3

```
    </section>  
    <p className="message">  
      준비 완료! 다음 단계에서 카드를 뒤집어요.  
    </p>  
  </main>  
);  
}
```

세 부분을 모두 이어 붙인 뒤 Ctrl + S로 저장해요. 중간에는 문법 오류가 보일 수 있어요.

08 App.css 1/7

```
* { box-sizing: border-box; }  
body {  
  margin: 0;  
  min-height: 100vh;  
  min-height: 100dvh;  
  background: #10172e;  
  color: #f5f5ff;  
  font-family: "Malgun Gothic", sans-serif;  
}  
button { font: inherit; }
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

08 App.css 2/7

```
.game {  
  width: 100%;  
  max-width: 440px;  
  margin: 0 auto;  
  padding: 28px 20px;  
}  
.game-header { text-align: center; }  
.eyebrow {  
  margin: 0 0 12px;  
  color: #8ff0cd;  
  font-size: 13px;  
  font-weight: bold;  
  letter-spacing: 3px;  
}
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

src/App.css / 기존 코드를 유지하고 맨 아래에 추가

08 App.css 3/7

```
h1 {  
  margin: 0;  
  font-size: clamp(23px, 6vw, 30px);  
  letter-spacing: -1px;  
}  
.description {  
  margin: 14px 0 24px;  
  color: #bac5e7;  
  font-size: 14px;  
  line-height: 1.6;  
}
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

08 App.css 4/7

```
.scoreboard {  
  display: flex;  
  justify-content: space-between;  
  gap: 10px;  
  margin-bottom: 20px;  
}  
.scoreboard p {  
  flex: 1;  
  margin: 0;  
  padding: 14px 8px;  
  border: 1px solid #394774;  
  border-radius: 14px;  
  background: #202d50;  
  text-align: center; font-size: 14px;  
}
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

src/App.css / 기존 코드를 유지하고 맨 아래에 추가

08 App.css 5/7

```
.scoreboard strong {  
  margin-left: 5px;  
  color: #8ff0cd;  
  font-size: 20px;  
}  
.card-grid {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
  gap: 14px;  
}
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

08 App.css 6/7

```
.card {  
  display: grid;  
  place-items: center;  
  width: 100%;  
  aspect-ratio: 1;  
  padding: 0;  
  border: 3px solid #505a98;  
  border-radius: 22px;  
  background: linear-gradient(145deg, #2c3965, #202949);  
  color: #eeeeff;  
  font-size: clamp(48px, 15vw, 72px);  
  font-weight: bold;  
  cursor: pointer;  
  touch-action: manipulation;  
}
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

08 App.css 7/7

```
.card:active {  
  transform: scale(0.97);  
}  
.card:focus-visible {  
  outline: 3px solid #8ff0cd;  
  outline-offset: 3px;  
}  
.message {  
  margin: 24px 0 0;  
  color: #d6c5ff;  
  text-align: center;  
  font-size: 14px;  
  line-height: 1.6;  
}
```

전체 일곱 부분을 같은 파일에 순서대로 이어 붙이고 저장해요.

09 컴퓨터에서 실행

`npm run dev -- --host 0.0.0.0`

터미널의 Local 주소를 Ctrl + 클릭해요.

카드 4장이 두 줄로 표시되는지 확인해요.



```
문제  출력  디버그 콘솔  터미널  포트

PS C:\Users\ej070\memory-game> npm install -D vite

found 0 vulnerabilities
PS C:\Users\ej070\memory-game> npm run dev -- --host 0.0.0.0

> memory-game@1.0.0 dev
> vite --host 0.0.0.0

VITE v8.3.1 ready in 1297 ms
  → Local:   http://localhost:5173/
  → Network: http://172.30.1.60:5173/ Wi-Fi
```

명령은 memory-game 폴더에서 실행해요. 이미 서버가 실행 중이면 Ctrl + C 후 실행해요.

10 휴대폰으로 접속

컴퓨터와 휴대폰을 같은 Wi-Fi에 연결해요.

터미널에서 Network 주소를 확인해요.

휴대폰 브라우저에 주소 전체를 입력해요.

예: `http://192.168.0.15:5173/`

예시 주소 대신 자신의 Network 주소를 사용해요. 포트 번호까지 입력하고 서버를 켜 두어요.

휴대폰 실행 결과

휴대폰에서 실제 실행한 화면이에요.

카드를 손가락으로 누를 수 있어요.

이번 단계에서는 놀림 효과만 보여요.

그림 공개 이벤트는 2단계에서 추가해요.



터치해서 카드 뒤집기

1단계 파일을 수정하는 실습

src/App.jsx 이벤트와 상태 추가
src/App.css 열린 카드 색상 추가



완성 목표: 터치한 카드 한 장의 그림과 번호가 화면에 나타나요.

이번 실습의 완성 화면

3번 카드를 누르면 그림이 보여요.

선택한 카드는 연보라색으로 바뀌어요.

아래 문구에도 카드 번호가 나타나요.

다른 카드를 누르면 이전 카드는 닫혀요.



01 useState 가져오기

찾을 기존 코드

```
import React from "react";
```

아래 한 줄로 교체

```
import React, { useState } from "react";
```

그다음 줄은 유지

```
import "./App.css";
```

useState는 선택한 카드 번호를 기억하고 화면에 반영하는 데 사용해요.

02 새 코드가 들어갈 위치

```
export default function App() {
```

```
  // 다음 장의 코드를 이 위치에 추가
```

```
  return (
```

위의 주석은 위치 안내예요. App 함수 밖이나 return 안에 추가하지 않아요.

02 카드 데이터와 상태 추가

```
const cards = [  
  { id: 1, emoji: "♂️" },  
  { id: 2, emoji: "♀️" },  
  { id: 3, emoji: "♂️" },  
  { id: 4, emoji: "♀️" }  
];  
const [selectedId, setSelectedId] = useState(null);  
  
function handleCardClick(id) {  
  setSelectedId(id);  
}
```

문자열의 ♂️는 마법사, ♀️는 용이에요. 그대로 입력하면 그림이 나와요.

상태와 이벤트 함수의 역할

`cards`: 카드 번호와 그림을 담은 목록

`selectedId`: 지금 선택한 카드 번호

`null`: 아직 아무 카드도 선택하지 않은 상태

`handleCardClick(3)`: 선택한 번호를 3으로 변경

`setSelectedId`가 상태를 바꾸면 React가 화면을 다시 계산해요.

03 교체할 카드 영역 찾기

시작 위치

```
<section className="card-grid" aria-label="게임 카드">
```

이 안에 있는 기존 button 4개를 모두 포함

끝 위치

```
</section>
```

시작 section부터 닫는 /section까지 선택해요. 점수판과 안내 문구는 제외해요.

03 카드 영역 교체 1/2

```
<section className="card-grid" aria-label="게임 카드">
  {cards.map((card) => (
    <button
      key={card.id}
      type="button"
      className={
        selectedId === card.id ? "card is-open" : "card"
      }
      onClick={() => handleCardClick(card.id)}
    >
```

다음 장 코드를 바로 이어 붙여야 완성돼요. 아직 닫는 태그를 추가하지 마세요.

03 카드 영역 교체 2/2

```
      aria-label={
        selectedId === card.id
          ? `${card.id}번 카드: ${card.emoji}`
          : `${card.id}번 카드 뒤집기`
      }
      aria-pressed={selectedId === card.id}
    >
      {selectedId === card.id ? card.emoji : "?"}
    </button>
  )))
</section>
```

두 장 코드를 합쳐 section 하나가 돼요. 기존 버튼 4개는 남아 있으면 안 돼요.

onClick과 조건부 표시

`onClick={() => handleCardClick(card.id)}`
터치하거나 클릭했을 때 해당 카드 번호를 전달해요.

`selectedId === card.id ? card.emoji : "?"`
선택한 번호와 같으면 그림, 다르면 ?를 표시해요.

`cards.map(...)`은 데이터 4개로 버튼 4개를 만들어요.

`key`는 각 카드를 구별하는 값이에요. `onClick`의 `C`는 반드시 대문자예요.

04 기존 안내 문구 선택

찾을 기존 코드

```
<p className="message">  
  준비 완료! 다음 단계에서 카드를 뒤집어요.  
</p>
```

이 p의 시작 태그부터 닫는 태그까지 선택해요. 다음 장 코드로 교체해요.

04 선택 번호가 표시되는 안내 문구

```
<p className="message" aria-live="polite">
  {selectedId === null
    ? "카드를 한 장 터치해 보세요."
    : `${selectedId}번 카드를 선택했어요!`}
</p>
```

`${selectedId}`가 들어간 문자열은 작은따옴표가 아닌 백틱(`)으로 감싸요.

05 열린 카드 디자인 추가

```
.card.is-open {  
  background: #e4d8ff;  
  border-color: #c4a7ff;  
  color: #202949;  
}
```

마지막 } 뒤에 새 줄을 만들고 추가해요. 기존 .card와 .message 코드는 유지해요.

확인 · 두 파일 모두 Ctrl + S

저장과 컴퓨터 실행

서버가 실행 중이면 브라우저에서 결과를 확인해요.

서버가 꺼져 있으면 memory-game 터미널에서 실행

```
npm run dev -- --host 0.0.0.0
```

터미널의 Local 주소를 열어 카드를 클릭해요.

수정 도중 생긴 오류는 모든 코드 입력과 저장을 마친 뒤 확인해요.

휴대폰에서 터치 확인

컴퓨터와 휴대폰을 같은 Wi-Fi에 연결해요.

터미널의 Network 주소 전체로 접속해요.

기존 화면을 열어 두었다면 새로고침해요.

컴퓨터와 서버를 켜 둔 채 카드를 터치해요.



두 장 선택하고 짝 비교하기

2단계 파일을 수정하는 실습

src/App.jsx 선택과 비교 기능
src/App.css 정답 카드 디자인



목표: 같은 그림을 찾으면 민트색으로 남고, 찾은 짝과 시도 횟수가 바뀌어요.

3단계 완성 화면

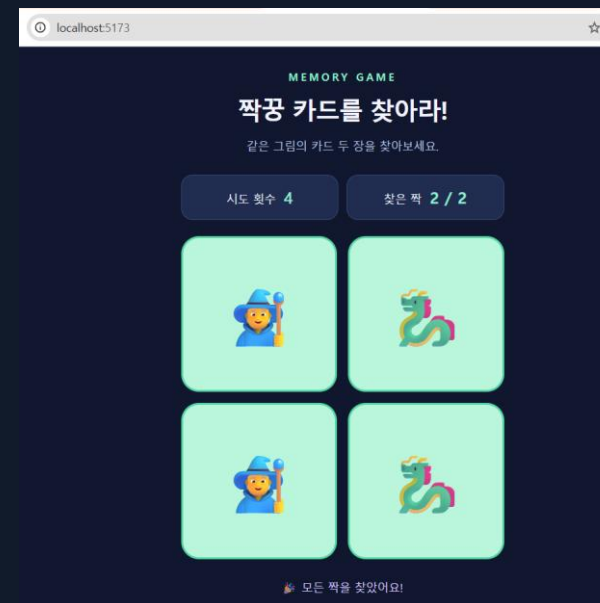
같은 그림 두 장을 선택하면 정답이에요.

맞춘 카드는 열린 상태로 남아요.

두 장을 비교할 때 시도 횟수가 늘어요.

모두 찾으면 완료 문구가 나타나요.

사진의 시도 횟수는 플레이에 따라 달라요.



01 교체할 기존 코드(삭제)

```
const [selectedId, setSelectedId] = useState(null);

function handleCardClick(id) {
  setSelectedId(id);
}
```

상태 선언부터 함수의 닫는 }까지 모두 선택해요. 아래 return (은 선택하지 않아요.

src/App.jsx · 선택한 기존 코드를 지운 자리에 입력

01 새 상태 3개

```
const [selectedIds, setSelectedIds] = useState([]);
```

```
const [matchedIds, setMatchedIds] = useState([]);
```

```
const [attempts, setAttempts] = useState(0);
```

이 코드 아래에 다음 세 장의 함수를 순서대로 이어 붙여요. 모두 return 위에 위치해요.

01 클릭 함수 1/3

```
function handleCardClick(id) {  
  if (matchedIds.includes(id)) {  
    return;  
  }  
  
  if (selectedIds.length === 2) {  
    setSelectedIds([id]);  
    return;  
  }  
}
```

맞춘 카드는 건너뛰고, 두 장 비교 후 누른 새 카드를 첫 번째 카드로 지정해요.

01 클릭 함수 2/3

```
if (selectedIds.includes(id)) {  
  return;  
}
```

```
const nextSelectedIds = [...selectedIds, id];  
setSelectedIds(nextSelectedIds);
```

```
if (nextSelectedIds.length === 2) {  
  setAttempts((prev) => prev + 1);  
}
```

같은 카드를 중복 선택하지 않아요. 두 번째 카드를 선택하면 시도 횟수를 늘려요.

01 클릭 함수 3/3

```
const firstCard = cards.find(
  (card) => card.id === nextSelectedIds[0]
);
const secondCard = cards.find(
  (card) => card.id === nextSelectedIds[1]
);
if (firstCard.emoji === secondCard.emoji) {
  setMatchedIds((prev) => [...prev, ...nextSelectedIds]);
}
}
```

마지막 }는 handleCardClick 함수의 끝이에요. 그다음에 기존 return (이 이어져요.

추가 위치 최종 확인

```
import 두 줄
export default function App() {
  기존 cards 배열
  새 상태 선언 3개
  새 handleClick 함수
  return (
    기존 화면 JSX
  );
}
```

이 장은 구조 안내예요. 화면에 적힌 "기존 cards 배열" 등을 코드에 입력하지 마세요.

세 가지 상태의 의미

`selectedIds`: 이번에 선택한 카드 번호, 예: [1, 3]

`matchedIds`: 지금까지 짝을 맞춘 카드 번호

`attempts`: 카드 두 장을 비교한 횟수

[]는 아직 카드 번호가 하나도 없는 빈 배열이에요.

`nextSelectedIds`는 방금 누른 번호까지 포함해요. 이 배열로 두 번째 선택을 판단해요.

02 기존 카드 영역 선택

시작 태그

```
<section className="card-grid" aria-label="게임 카드">
```

안쪽의 cards.map 코드와 button 전체 포함

끝 태그

```
</section>
```

시작부터 끝까지 지우고 다음 두 장의 코드를 순서대로 입력해요.

02 카드 영역 1/2

```
<section className="card-grid" aria-label="게임 카드">
  {cards.map((card) => {
    const isMatched = matchedIds.includes(card.id);
    const isOpen = selectedIds.includes(card.id) || isMatched;
    return (
      <button
        key={card.id}
        type="button"
        className={
          isMatched ? "card is-matched"
            : isOpen ? "card is-open" : "card"
        }
      )
    )
  })}
```

선택한 카드 또는 이미 맞춘 카드가 열려요. 다음 장을 이어 입력하세요.

02 카드 영역 2/2

```
    onClick={() => handleCardClick(card.id)}
    disabled={isMatched}
    aria-label={isOpen
      ? `${card.id}번 카드: ${card.emoji}`
      : `${card.id}번 카드 뒤집기`}
    aria-pressed={isOpen}
  >
    {isOpen ? card.emoji : "?"}
  </button>
);
}}
</section>
```

맞춘 카드는 disabled로 재선택을 막아요. 닫는 괄호와 태그까지 모두 입력하세요.

03 기존 점수판 선택

```
<div className="scoreboard">  
  <p>시도 횟수 <strong>0</strong></p>  
  <p>찾은 짝 <strong>0 / 2</strong></p>  
</div>
```

이 div 전체를 선택한 뒤 다음 장의 코드로 교체해요.

03 상태로 표시하는 점수판

```
<div className="scoreboard">
  <p>
    시도 횟수 <strong>{attempts}</strong>
  </p>
  <p>
    찾은 짝{" "}
    <strong>
      {matchedIds.length / 2} / {cards.length / 2}
    </strong>
  </p>
</div>
```

맞춘 카드가 2장이면 찾은 짝은 1이에요. 시도 횟수는 두 장 선택할 때 1 증가해요.

04 기존 안내 문구 선택

```
<p className="message" aria-live="polite">
  {selectedId === null
    ? "카드를 한 장 터치해 보세요."
    : `${selectedId}번 카드를 선택했어요!`}
</p>
```

이 p의 시작 태그부터 끝 태그까지 다음 장 코드로 교체해요.

04 상황별 안내 문구

```
<p className="message" aria-live="polite">
  {matchedIds.length === cards.length
    ? "모든 짝을 찾았어요!"
    : selectedIds.length === 0
      ? "첫 번째 카드를 선택하세요."
      : selectedIds.length === 1
        ? "두 번째 카드를 선택하세요."
        : selectedIds.every((id) => matchedIds.includes(id))
          ? "정답! 다음 카드를 선택하세요."
          : "다른 그림이에요! 다음 카드를 누르면 다시 시작해요."}
</p>
```

기존 selectedId를 사용하는 코드가 남아 있으면 오류가 나요. 새 이름은 selectedIds예요.

05 정답 카드 스타일 추가

```
.card.is-matched {  
  background: #b9f6dc;  
  border-color: #52d6a4;  
  color: #163d32;  
  cursor: default;  
}
```

```
.card.is-matched:active {  
  transform: none;  
}
```

기존 마지막 } 다음 줄에 추가해요. 기존 .card와 .card.is-open 스타일은 유지해요.

확인 · 두 파일 모두 Ctrl + S

저장과 실행

서버가 실행 중이면 브라우저를 새로고침해요.

서버가 꺼져 있으면 memory-game 터미널에서 실행

```
npm run dev -- --host 0.0.0.0
```

휴대폰은 같은 Wi-Fi에서 Network 주소로 접속해요.

모든 수정이 끝난 뒤 확인해요. 입력 중에는 이전 변수 이름 때문에 오류가 보일 수 있어요.

휴대폰 완성 결과

사용자가 실행한 휴대폰 화면이에요.

1·3번, 2·4번을 바로 맞추면 시도 2예요.

맞춘 네 장 모두 민트색으로 남아요.

화면의 축하 이모지는 선택 사항이에요.

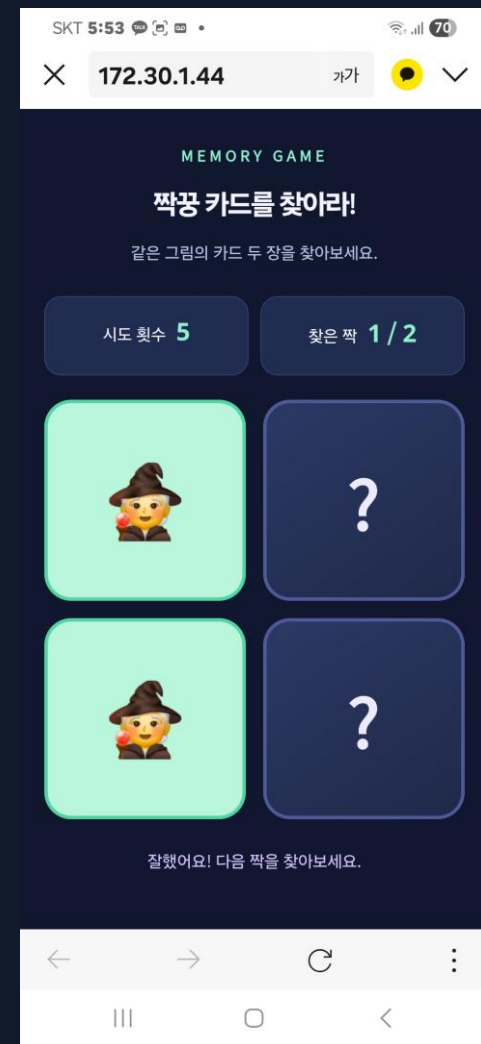
다시 해보려면 브라우저를 새로고침해요.



1초 뒤 자동으로 카드 닫기

3단계 파일을 수정하는 실습

다른 그림 두 장을 잠깐 보여주기
기다리는 동안 다른 카드 선택 막기



수정 파일: src/App.jsx

이번 단계의 동작

다른 그림 두 장 선택: 두 장을 보여주고 선택 잠금

약 1초 뒤: 두 장을 닫고 선택 잠금 해제

같은 그림 두 장 선택: 민트색으로 유지

맞춘 카드가 있어도 남은 카드로 계속 진행

기다리는 동안 세 번째 카드를 눌러도 열리지 않아야 해요.

01 잠금 상태 추가 위치

찾을 기존 코드

```
const [attempts, setAttempts] = useState(0);
```

이 줄 바로 아래에 추가

```
const [isLocked, setIsLocked] = useState(false);
```

App 함수 안, handleClick 함수 위에 추가해요. 기존 상태 선언은 유지해요.

02 클릭 함수 맨 위에 추가

```
function handleClick(id) {  
  if (isLocked) {  
    return;  
  }  
  
  if (matchedIds.includes(id)) {  
    return;  
  }  
}
```

추가하는 부분은 if (isLocked) 블록이에요. 아래 matchedIds 검사부터는 기존 코드예요.

03 기존 초기화 코드 삭제

```
if (selectedIds.length === 2) {  
  setSelectedIds([id]);  
  return;  
}
```

이 if문의 시작부터 닫는 }까지 삭제해요. 다른 if문은 지우지 마세요.

04 교체할 그림 비교 코드

```
if (firstCard.emoji === secondCard.emoji) {  
  setMatchedIds((prev) => [...prev, ...nextSelectedIds]);  
}
```

이 if문 전체를 다음 장 코드로 교체해요. 그 아래의 바깥쪽 닫는 } 두 개는 유지해요.

04 자동 닫기 코드로 교체

```
if (firstCard.emoji === secondCard.emoji) {  
  setMatchedIds((prev) => [...prev, ...nextSelectedIds]);  
  setSelectedIds([]);  
} else {  
  setIsLocked(true);  
  
  setTimeout(() => {  
    setSelectedIds([]);  
    setIsLocked(false);  
  }, 1000);  
}
```

1000밀리초는 1초예요. 타이머 안에서 선택 목록을 비우고 잠금을 풀어요.

정답 카드는 계속 열려 있어요

`setSelectedIds([])`

이번에 선택한 카드 번호 목록을 비워요.

`matchedIds`에는 맞춘 카드 번호가 남아 있어요.

기존 `isOpen` 조건이 `matchedIds`도 확인하므로
맞춘 카드는 계속 열린 상태를 유지해요.

정답이면 바로 다음 선택을 준비하고, 오답이면 타이머가 선택 목록을 비워요.

05 버튼 disabled 한 줄 교체

찾을 기존 코드

```
disabled={isMatched}
```

교체할 코드

```
disabled={isMatched || isLocked}
```

카드 section 안의 button 속성이에요. className, onClick, isOpen 코드는 유지해요.

06 안내 문구 교체 범위

시작 태그

```
<p className="message" aria-live="polite">
```

안쪽의 기존 조건식 전체 포함

끝 태그

```
</p>
```

시작 p부터 닫는 /p까지 선택하고 다음 장 코드로 교체해요.

06 자동 닫기 안내 문구

```
<p className="message" aria-live="polite">
  {matchedIds.length === cards.length
    ? "모든 짝을 찾았어요!"
    : isLocked
      ? "다른 그림이에요! 잠시 후 카드가 닫혀요."
      : selectedIds.length === 1
        ? "두 번째 카드를 선택하세요."
        : matchedIds.length > 0
          ? "잘했어요! 다음 짝을 찾아보세요."
          : "첫 번째 카드를 선택하세요."}
</p>
```

축하 이모지는 선택 사항이에요. 기존 "다음 카드를 누르면" 문구는 남기지 않아요.

확인용 · 이미 수정했다면 다시 추가하지 않아요

함수 전체 비교 1/3

```
function handleCardClick(id) {  
  if (isLocked) {  
    return;  
  }  
  if (matchedIds.includes(id)) {  
    return;  
  }  
  if (selectedIds.includes(id)) {  
    return;  
  }  
}
```

다음 두 장과 이어지는 함수 하나예요. isLocked 검사부터 시작하는지 확인해요.

함수 전체 비교 2/3

```
const nextSelectedIds = [...selectedIds, id];
setSelectedIds(nextSelectedIds);
if (nextSelectedIds.length === 2) {
  setAttempts((prev) => prev + 1);
  const firstCard = cards.find(
    (card) => card.id === nextSelectedIds[0]
  );
  const secondCard = cards.find(
    (card) => card.id === nextSelectedIds[1]
  );
```

두 번째 카드를 선택한 경우에만 시도 횟수를 늘리고 두 그림을 비교해요.

함수 전체 비교 3/3

```
if (firstCard.emoji === secondCard.emoji) {  
  setMatchedIds((prev) => [...prev, ...nextSelectedIds]);  
  setSelectedIds([]);  
} else {  
  setIsLocked(true);  
  setTimeout(() => {  
    setSelectedIds([]);  
    setIsLocked(false);  
  }, 1000);  
}  
}  
}
```

함수는 하나만 있어야 해요. 이 마지막 } 다음에 App의 기존 return (이 이어져요.

확인 · src/App.jsx에서 Ctrl + S

저장과 실행

저장한 뒤 브라우저를 새로고침해요.

서버가 꺼져 있으면 memory-game 터미널에서 실행

```
npm run dev -- --host 0.0.0.0
```

휴대폰은 같은 Wi-Fi에서 현재 Network 주소로 접속해요.

이번 단계는 CSS 수정이 없어요. 이전 단계의 남색·연보라색·민트색 디자인을 사용해요.

확인 1 · 자동으로 닫히는지

위 왼쪽 1번과 위 오른쪽 2번을 눌러요.

다른 그림 두 장이 보이고 시도 횟수는 1이에요.

다른 곳을 누르지 않고 약 1초 기다려요.

두 장이 모두 ?로 돌아오면 성공이에요.

그다음 카드를 눌렀을 때 다시 열리는지도 확인해요.

실습 · 새로그침 후 1, 2, 3번을 빠르게 터치

확인 2 · 연속 터치와 정답

1번, 2번을 누른 직후 3번을 눌러요.
기다리는 동안 3번이 열리지 않아야 해요.

두 장이 닫힌 뒤 1번과 3번을 눌러요.
마법사 두 장이 민트색으로 계속 남아야 해요.

2번과 4번을 누르면 완료 문구가 나타나요.

같은 카드를 두 번 눌러도 시도 횟수가 늘지 않아야 해요.

휴대폰의 진행 화면

첨부한 실제 휴대폰 화면이에요.

맞춘 마법사 두 장이 민트색으로 남아요.

다음 짝을 찾으라는 문구가 보여요.

시도 횟수는 선택 과정에 따라 달라요.

자동 닫기와 잠금은 직접 터치해 확인해요.



모든 짝을 찾은 화면

첨부한 컴퓨터 완료 화면이에요.

네 장 모두 민트색으로 남아요.

찾은 짝은 2 / 2예요.

사진의 시도 횟수 4는 플레이 결과예요.

다시 연습하려면 새로고침하세요.



카드 섞기와 다시 시작

4단계 파일을 수정하는 실습

src/App.jsx 카드 순서와 재시작 기능

src/App.css 재시작 버튼 디자인

목표: 새로 시작할 때 카드를 섞고 점수와 선택 기록을 초기화해요.

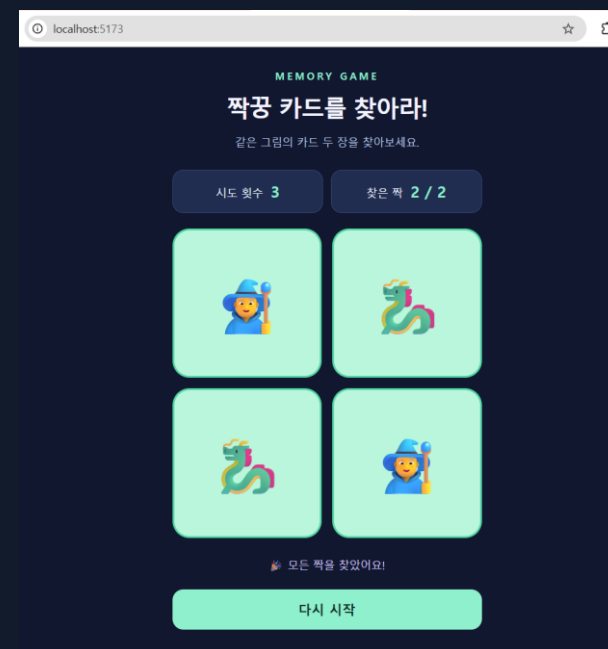
최종 완성 화면

첨부한 컴퓨터 실행 화면이에요.

그림의 배치가 이전 단계와 달라졌어요.

안내 문구 아래에 다시 시작 버튼이 있어요.

게임 도중과 완료 후 모두 재시작할 수 있어요.



01 섞기 함수가 들어갈 위치

```
import React, { useState } from "react";  
import "./App.css";
```

// 이 위치에 다음 장의 shuffleCards 함수 추가

```
export default function App() {
```

두 import 아래, export default function App() 위에 새 줄을 만들어요.

01 카드 섞기 함수 추가

```
function shuffleCards(cardList) {  
  const shuffled = [...cardList];  
  for (let i = shuffled.length - 1; i > 0; i--) {  
    const j = Math.floor(Math.random() * (i + 1));  
    const temp = shuffled[i];  
    shuffled[i] = shuffled[j];  
    shuffled[j] = temp;  
  }  
  return shuffled;  
}
```

원본 배열을 복사하고 두 카드의 위치를 바꿉요. id와 그림은 바꾸지 않아요.

02 교체할 기존 카드 배열

```
const cards = [  
  { id: 1, emoji: "🍷" },  
  { id: 2, emoji: "🍷" },  
  { id: 3, emoji: "🍷" },  
  { id: 4, emoji: "🍷" }  
];
```

이모지 그림으로 입력한 코드도 같은 배열이에요. `const cards`부터 `];`까지 전체 선택해요.

02 카드 배열을 상태로 교체

```
const [cards, setCards] = useState(() =>
  shuffleCards([
    { id: 1, emoji: "🔮" },
    { id: 2, emoji: "🐉" },
    { id: 3, emoji: "🔮" },
    { id: 4, emoji: "🐉" }
  ])
);
```

문자열의 🔮는 마법사, 🐉는 용이에요. 그대로 입력하면 그림이 나와요.

카드 상태의 역할

cards: 지금 화면에 배치할 카드 목록

setCards: 새로운 순서를 화면에 반영하는 함수

useState() => shuffleCards(...)
게임을 처음 열 때 사용할 카드 순서를 만들어요.

터치할 때마다 섞지 않아요. 이후에는 다시 시작 버튼을 눌렀을 때 섞어요.

03 재시작 함수가 들어갈 위치

```
const [selectedIds, setSelectedIds] = useState([]);  
const [matchedIds, setMatchedIds] = useState([]);  
const [attempts, setAttempts] = useState(0);  
const [isLocked, setIsLocked] = useState(false);
```

// 이 위치에 다음 장의 handleRestart 함수 추가

```
function handleCardClick(id) {
```

기존 상태 네 줄은 유지해요. isLocked 선언 아래, handleCardClick 위에 추가해요.

03 다시 시작 함수 추가

```
function handleRestart() {  
  if (isLocked) {  
    return;  
  }  
  setCards(shuffleCards(cards));  
  setSelectedIds([]);  
  setMatchedIds([]);  
  setAttempts(0);  
  setIsLocked(false);  
}
```

자동 닫기를 기다리는 1초 동안은 재시작을 막아요. 이전 타이머와 새 게임이 겹치지 않아요.

확인 · 구조 안내이므로 이 코드를 붙여 넣지 않아요

두 함수의 위치 비교

```
import 두 줄  
shuffleCards 함수          ← App 밖  
  
export default function App() {  
  cards 상태와 기존 상태 네 줄  
  handleRestart 함수        ← App 안  
  기존 handleClick 함수  
  return (기존 화면 JSX);  
}
```

shuffleCards는 카드를 섞어요. handleRestart는 App의 상태를 초기화해요.

04 버튼 추가 위치 찾기

```
</p>
```

```
// 안내 문구의 닫는 p와 닫는 main 사이  
// 다음 장의 button 전체를 이 위치에 추가
```

```
</main>
```

```
);
```

```
}
```

이 장의 // 주석은 위치 안내예요. JSX에 입력하지 말고 다음 장의 버튼만 추가하세요.

04 재시작 버튼 추가

```
<button  
  type="button"  
  className="restart-button"  
  onClick={handleRestart}  
  disabled={isLocked}  
>  
  {isLocked ? "잠시만 기다려 주세요..." : "다시 시작"}  
</button>
```

기존 안내 문구를 지우지 않아요. 버튼은 카드 section 밖, main 안에 위치해요.

버튼 이벤트와 잠금

`onClick={handleRestart}`

버튼을 누르면 재시작 함수를 실행해요.

`disabled={isLocked}`

자동 닫기를 기다리는 동안 버튼을 비활성화해요.

함수 안의 `if (isLocked)`도 재시작을 막아요.

`onClick={handleRestart()}`로 쓰면 렌더링 중 실행돼요. 함수 이름만 전달하세요.

src/App.css · Ctrl + End, 기존 마지막 } 아래에 추가

05 버튼 CSS 추가 1/2

```
.restart-button {  
  display: block;  
  width: 100%;  
  min-height: 52px;  
  margin-top: 20px;  
  padding: 14px 18px;  
  border: none;  
  border-radius: 14px;  
  background: #8ff0cd;  
  color: #102b25;
```

다음 장 코드를 바로 이어 입력해요. 아직 .restart-button의 닫는 }를 추가하지 마세요.

05 버튼 CSS 추가 2/2

```
font-size: 17px;  
font-weight: bold;  
cursor: pointer;  
touch-action: manipulation;  
}  
.restart-button:active:not(:disabled) {  
  transform: scale(0.98);  
}  
.restart-button:disabled {  
  opacity: 0.55;  
  cursor: not-allowed;  
}
```

활성 버튼을 누르면 살짝 작아지고, 잠긴 버튼은 흐리게 표시돼요.

src/App.css · 앞 장 마지막 } 다음 줄에 추가

05 키보드 선택 표시 추가

```
.restart-button:focus-visible {  
  outline: 3px solid #ffffff;  
  outline-offset: 3px;  
}
```

버튼에 키보드 초점이 오면 테두리를 표시해요. 기존 카드 CSS는 그대로 유지해요.

확인 · App.jsx와 App.css 모두 Ctrl + S

저장과 실행

저장 후 브라우저를 새로고침해요.

서버가 꺼져 있으면 memory-game 터미널에서 실행

```
npm run dev -- --host 0.0.0.0
```

휴대폰은 같은 Wi-Fi에서 현재 Network 주소로 접속해요.

버튼이 화면 아래에 가려지면 스크롤해 확인해요.

확인 1 · 게임 도중 재시작

카드 한 장을 연 뒤 다시 시작을 눌러요.
모든 카드가 ?로 돌아오는지 확인해요.

한 쌍을 맞춘 뒤 다시 시작을 눌러요.
맞춘 민트색 카드도 닫히는지 확인해요.

시도 횟수 0, 찾은 짝 0 / 2인지 확인해요.

이제 그림 위치가 무작위예요. 이전 단계처럼 특정 위치의 그림을 가정하지 마세요.

확인 2 · 재시작 잠금

첨부한 휴대폰의 대기 화면이에요.

다른 그림 두 장을 열면 버튼이 흐려져요.

“잠시만 기다려 주세요...”가 표시돼요.

약 1초 뒤 다시 시작 버튼이 활성화돼요.

활성화 후 재시작이 되는지 확인해요.



확인 3 · 완료 후 재시작과 섞기

모든 짝을 맞춘 뒤 다시 시작을 눌러요.
완료 문구가 사라지고 카드가 닫혀야 해요.

다시 뒤집어서 카드 배치를 확인해요.

우연히 이전과 같은 배치가 나올 수 있어요.
여러 번 플레이하며 확인하세요.

섞어도 각 그림은 두 장씩 남고, 카드 id는 바뀌지 않아야 해요.

2_5 이벤트 속성 이해하기

이벤트 속성은 사용자 동작에 실행할 함수를 연결해요.

카드를 터치하면 카드 선택 함수를 실행해요.

‘다시 시작’을 누르면 게임 초기화 함수를 실행해요.

```
<button onClick={handleRestart}>  
  다시 시작  
</button>
```

오늘의 목표: 사용자 동작과 함수, 상태 변경의 관계를 이해해요.

이벤트와 처리 함수의 관계

구분	의미	게임 예시
이벤트	사용자 동작 등으로 발생하는 일	카드를 클릭함
이벤트 속성	이벤트에 함수를 연결하는 속성	onClick
이벤트 처리 함수	이벤트 발생 시 실행하는 함수	handleCardClick

휴대폰에서 버튼을 터치하는 동작도 일반적으로 onClick으로 처리해요.

함수를 만들고 버튼에 연결하기

```
function App() {  
  function handleClick() {  
    alert("카드를 클릭했어요!");  
  }  
  
  return (  
    <button onClick={handleClick}>카드 열기</button>  
  );  
}  
  
export default App;
```

버튼 클릭 → handleClick 실행 → 알림창 표시

함수 전달과 함수 호출

클릭할 때 실행하도록 함수 전달

```
<button onClick={handleClick}>카드 열기</button>
```

handleClick은 함수 자체를 가리켜요.

화면을 만드는 도중 즉시 호출

```
<button onClick={handleClick()}>카드 열기</button>
```

이 예제의 이벤트 속성에는 handleClick처럼 나중에 실행할 함수를 전달해요.

카드 번호를 전달하는 방법

```
<button  
  onClick={() => handleCardClick(card.id)}  
>  
  ?  
</button>
```

() => ... 는 클릭했을 때 실행할 함수예요.

그 함수 안에서 handleCardClick(card.id)를 호출해요.

card.id가 3이면 클릭 시 handleCardClick(3)을 실행해요.

onClick={handleCardClick(card.id)}로 쓰면 렌더링 중에 바로 호출돼요.

이벤트와 상태의 역할

이벤트: 연결된 함수를 실행하는 계기

상태: 화면에 반영할 데이터를 기억하는 수단

setIsOpen(true): 상태를 바꾸고 새 화면 렌더링을 요청

```
{isOpen ? "열림" : "?"}
```

onClick만으로 화면이 바뀌지는 않아요. 처리 함수에서 할 일을 정해야 해요.

클릭하면 열리는 카드

```
import { useState } from "react";

function App() {
  const [isOpen, setIsOpen] = useState(false);
  function handleClick() {
    setIsOpen(true);
  }
  return (
    <button onClick={handleClick}>
      {isOpen ? "열림" : "?"}
    </button>
  );
}
export default App;
```

처음에는 ? 표시 → 클릭 → 상태 변경 요청 → 열림 표시

자주 사용하는 이벤트 속성

속성	발생하는 상황	활용 예시
onClick	클릭하거나 버튼을 터치할 때	카드 선택, 다시 시작
onChange	입력값이 바뀔 때	플레이어 이름 입력
onKeyDown	키보드 키를 누를 때	Enter 키 처리
onSubmit	폼을 제출할 때	입력 후 게임 시작
onPointerDown	포인터로 누르기 시작할 때	누르는 순간 반응

대소문자를 구분해요. onclick이 아니라 onClick으로 작성해요.

이벤트 객체로 입력값 읽기

```
function handleChange(e) {  
  console.log(e.target.value);  
}
```

```
<input onChange={handleChange} />
```

e: React가 처리 함수에 전달하는 이벤트 정보

e.target: 이벤트가 발생한 입력 요소

e.target.value: 입력창에 들어 있는 현재 값

입력값은 개발자 도구의 콘솔에 표시돼요. e 대신 event라는 이름도 가능해요.

우리 게임 코드에서 확인하기

```
onClick={() => handleCardClick(card.id)}
```

```
onClick={handleRestart}
```

- ① 카드 버튼에서 onClick 속성을 찾아보세요.
- ② handleCardClick 함수가 바꾸는 상태를 찾아보세요.
- ③ 다시 시작 버튼과 handleRestart의 연결을 확인하세요.

설명 연습: '이 버튼을 누르면 어떤 함수가 실행되고, 어떤 화면이 바뀔까?'